Aplicaciones Móviles



DESARROLLO DE APLICACIONES EN ANDROID

IMPLEMENTACIÓN DE LIBRERÍA SWEETALERT DIALOG

IMPLEMENTACIÓN DE UX - SWEETALERTDIALOG

Se implementará una nueva Activity en Android cuyo propósito será la demostración práctica del uso de alertas personalizadas mediante la librería **SweetAlertDialog**. Esta librería permite mostrar mensajes visualmente atractivos y diferenciados, mejorando la experiencia de usuario (UX) en las notificaciones dentro de la aplicación.

SweetAlertDialog ofrece diferentes tipos de mensajes (advertencia, éxito, error, progreso, confirmación, entre otros) que permiten estandarizar el comportamiento y diseño de las alertas en la aplicación, asegurando una interfaz más moderna y coherente.

Se diseñará un Activity que contenga **seis botones**, cada uno representando un tipo de alerta específica. Los botones deberán respetar la distribución, formato, color y texto tal como se muestra en la imagen proporcionada:



A Continuación, se declaran las variables correspondientes a los seis botones que compondrán la interfaz gráfica de la nueva Activity.

Cada variable es de tipo Button y se define utilizando el modificador lateinit, lo que indica que su inicialización se realizará posteriormente dentro del método onCreate(). El objetivo de esta declaración es preparar los identificadores necesarios para asociar, configurar y controlar el comportamiento de cada botón

```
private lateinit var btn1: Button
private lateinit var btn2: Button
private lateinit var btn3: Button
private lateinit var btn4: Button
private lateinit var btn5: Button
private lateinit var btn6: Button
```

En esta sección se realiza el enlace (cast) entre los componentes visuales definidos en el diseño XML y las variables declaradas previamente en el MainActivity.

Mediante el uso de findViewById(), se asocian los identificadores (id) de cada botón en la interfaz gráfica a sus respectivas variables (btn1 a btn6).

Este proceso es necesario para poder manipular los botones desde el código, asignarles eventos

```
btn1 = findViewById(R.id.button)
btn2 = findViewById(R.id.button2)
btn3 = findViewById(R.id.button3)
btn4 = findViewById(R.id.button4)
btn5 = findViewById(R.id.button5)
btn6 = findViewById(R.id.button6)
```

En este bloque se generan los eventos de escucha (OnClickListener) para cada uno de los seis botones previamente inicializados.

Cada botón se configura para que, al ser presionado por el usuario, ejecute un método específico que despliega el tipo de alerta correspondiente mediante SweetAlertDialog.

Esta implementación permite separar la lógica de cada tipo de alerta en métodos

independientes, favoreciendo la organización del código, su mantenimiento y la escalabilidad de la aplicación.

```
btn1.setOnClickListener {
   mostrarAdvertencia()
}

btn2.setOnClickListener{
   mostrarExito()
}

btn3.setOnClickListener{
   mostrarError()
}

btn4.setOnClickListener{
   mostrarCargando()
}

btn5.setOnClickListener{
   mostrarConfirmacion()
}

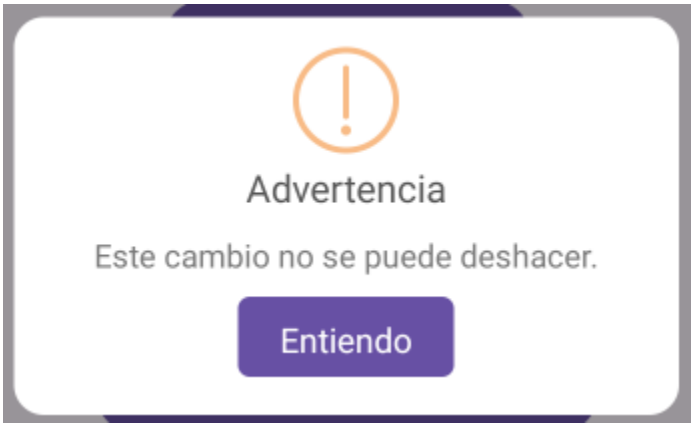
btn6.setOnClickListener{
   procesoConCargandoYExito()
}
```

A continuación, se procederá a la creación de los métodos individuales que gestionarán y desplegarán los distintos tipos de alertas que se desean ejemplificar.

Cada método encapsulará la configuración específica de una alerta utilizando la librería SweetAlertDialog, definiendo propiedades como el tipo de alerta, el título, el mensaje y el comportamiento del diálogo.

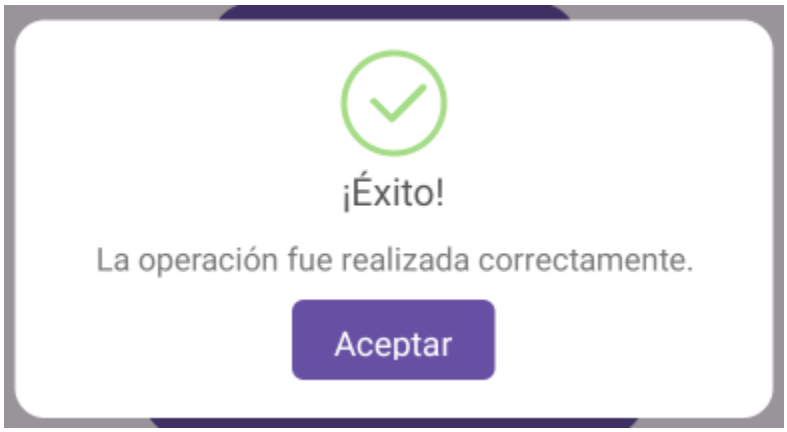
Esta estructuración modular facilita la reutilización del código, mejora su legibilidad y permite una fácil adaptación en caso de que se requieran modificaciones futuras en la presentación de las alertas.

EJEMPLO
⚠️ ADVERTENCIA (WARNING)




```
private fun mostrarAdvertencia() {
    SweetAlertDialog(this, SweetAlertDialog.WARNING_TYPE)
        .setTitleText("Advertencia")
        .setContentText("Este cambio no se puede deshacer.")
        .setConfirmText("Entiendo")
        .setConfirmClickListener { dialog ->
            dialog.dismissWithAnimation()
        }
        .show()
}
```

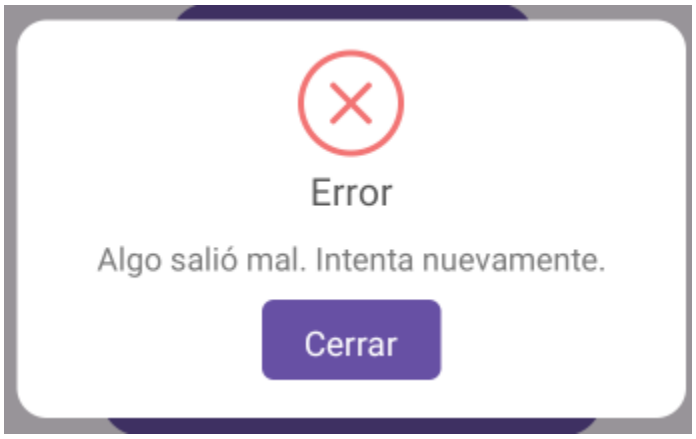
EJEMPLO
✅ MENSAJE DE ÉXITO (SUCCESS)



```
private fun mostrarExito() {
    SweetAlertDialog(this, SweetAlertDialog.SUCCESS_TYPE)
        .setTitleText("¡Éxito!")
        .setContentText("La operación fue realizada correctamente.")
        .setConfirmText("Aceptar")
        .setConfirmClickListener { dialog ->
            dialog.dismissWithAnimation()
        }
        .show()
}
```

EJEMPLO

❌ MENSAJE DE ERROR (ERROR)




```
private fun mostrarError() {
    SweetAlertDialog(this, SweetAlertDialog.ERROR_TYPE)
        .setTitleText("Error")
        .setContentText("Algo salió mal. Intenta nuevamente.")
        .setConfirmText("Cerrar")
        .setConfirmClickListener { dialog ->
            dialog.dismissWithAnimation()
        }
        .show()
}
```

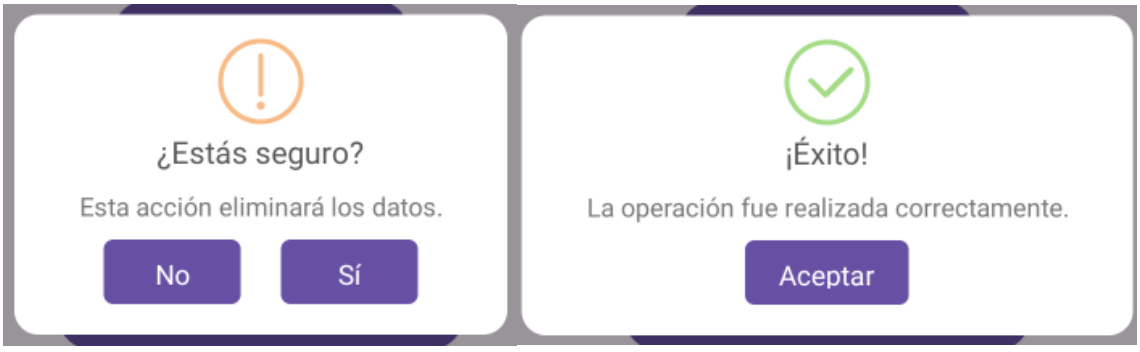
EJEMPLO
⏳ MENSAJE DE PROGRESO (CARGANDO)




```
private fun mostrarCargando()
{
   val pDialog = SweetAlertDialog(this, SweetAlertDialog.PROGRESS_TYPE)
   pDialog.titleText = "Cargando..."
   pDialog.setCancelable(false)
   pDialog.show()

   Handler(Looper.getMainLooper()).postDelayed({
      pDialog.dismissWithAnimation()
   }, 3000) // 3000 milisegundos = 3 segundos
}
```

EJEMPLO
🎯 CONFIRMACIÓN CON "SÍ" Y "NO"



```
private fun mostrarConfirmacion() {
    SweetAlertDialog(this, SweetAlertDialog.WARNING_TYPE)
        .setTitleText("¿Estás seguro?")
        .setContentText("Esta acción eliminará los datos.")
        .setConfirmText("Sí")
        .setCancelText("No")
        .setConfirmClickListener { dialog ->
            dialog.dismissWithAnimation()
            mostrarExito() // Aquí podrías ejecutar la acción real
        }
        .setCancelClickListener { dialog ->
            dialog.dismissWithAnimation()
        }
        .show()
}
```

EJEMPLO
CARGANDO...CAMBIAR A ¡ÉXITO!




```
private fun procesoConCargandoYExito() {
    val pDialog = SweetAlertDialog(this, SweetAlertDialog.PROGRESS_TYPE)
    pDialog.titleText = "Cargando..."
    pDialog.setCancelable(false)
    pDialog.show()

    // Simular una operación (por ejemplo, guardar datos)
    Handler(Looper.getMainLooper()).postDelayed({
        pDialog.changeAlertType(SweetAlertDialog.SUCCESS_TYPE)
        pDialog.titleText = "¡Éxito!"
        pDialog.contentText = "La operación se completó correctamente."
```

```
            pDialog.confirmText = "Aceptar"
            pDialog.setConfirmClickListener { dialog ->
                dialog.dismissWithAnimation()
            }
        }, 3000) // Espera 3 segundos simulando un proceso
    }
```

Para utilizar la librería SweetAlertDialog (u otras librerías hospedadas en JitPack), es necesario indicar al sistema de construcción de Android Studio (Gradle) que incluya el repositorio de JitPack como una fuente válida desde la cual descargar dependencias.

Esto se logra añadiendo la URL https://jitpack.io dentro del bloque de repositorios (repositories) en el archivo **settings.gradle** o, dependiendo de la estructura del proyecto, en el **build.gradle** a nivel de proyecto.

maven(url = "https://jitpack.io")

```
dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        google()
        mavenCentral()
        maven(url = "https://jitpack.io")
    }
}
```

¿Qué se está haciendo exactamente?

- Se está indicando a Gradle que, además de buscar dependencias en google() y mavenCentral(), también puede buscar en JitPack.

- Esto es necesario porque SweetAlertDialog y muchas otras librerías de proyectos comunitarios no se encuentran en los repositorios oficiales, sino que se publican directamente desde GitHub usando JitPack.

Una vez configurado el repositorio de JitPack en el settings.gradle, se procede a incluir la dependencia específica del proyecto que se utilizará, en este caso la librería SweetAlertDialog.

Para ello, en el archivo **build.gradle** (nivel módulo, normalmente **build.gradle** (Module: app)), dentro del bloque dependencies, se debe agregar la siguiente instrucción:

implementation("com.github.f0ris.sweetalert:library:1.6.2")

```
implementation("com.github.f0ris.sweetalert:library:1.6.2")
implementation(libs.androidx.core.ktx)
implementation(libs.androidx.appcompat)
implementation(libs.material)
```

¿Qué se está haciendo exactamente?

- Se está indicando que se desea incorporar en el proyecto la versión **1.6.2** de la librería SweetAlertDialog, publicada por el autor **f0ris** en GitHub.

IMPORTANTE: Debes sincronizar tanto el Settings como el Build